

Tugas Praktikum OOP - Modul 3

Encapsulation, Access Modifier, Getter–Setter, dan Validasi Data

Nama : Rafael Rizky
NRP : 3125522007

Dosen : Nirwana Haidar Hari, S.Pd., M.Kom.
Kampus : PENS PSDKU Sumenep (2026)

HALAMAN 1 — ANALISIS, SPRINT PLANNING & AUDIT P2

1. **Nama Project:** Sistem Kasir Sederhana
2. **Sprint Goal:** Memperbaiki struktur class proyek dengan menerapkan encapsulation sehingga seluruh data object bersifat private dan hanya dapat diakses/diubah melalui mekanisme yang terkontrol (Getter, Setter, dan Validasi).

3. Sprint Backlog (P3)

ID	Sprint Backlog	Status
SB-01	Mengubah seluruh attribute menjadi private pada semua class	DONE
SB-02	Membuat method getter untuk pembacaan data secara aman	DONE
SB-03	Membuat method setter selektif hanya untuk data yang dapat diubah	DONE
SB-04	Menambahkan validasi data (harga > 0, stok >= 0, format no HP, tipe member, stok transaksi)	DONE
SB-05	Memperbaiki constructor agar mematuhi aturan validasi	DONE
SB-06	Melakukan pengujian komprehensif (Test Valid dan Test Invalid)	DONE

4. Tabel Audit Class P2

Class	Attribute	Kondisi P2	Perbaikan P3
Produk	kode	default	private + getter only
Produk	nama	default	private + getter + setter (non-empty)
Produk	harga	default	private + getter + setter (> 0)
Produk	stok	default	private + getter + setter (>= 0)
Pelanggan	idPelanggan	default	private + getter only
Pelanggan	nama	default	private + getter + setter (non-empty)
Pelanggan	nomorHP	default	private + getter + setter (regex 10-13 digit)
Pelanggan	tipeMember	default	private + getter + setter (VIP/GOLD/REG)
Transaksi	idTransaksi, tgl	default	private + getter only
Transaksi	jumlahBeli	default	private + getter + setter (>0 & <= stok)

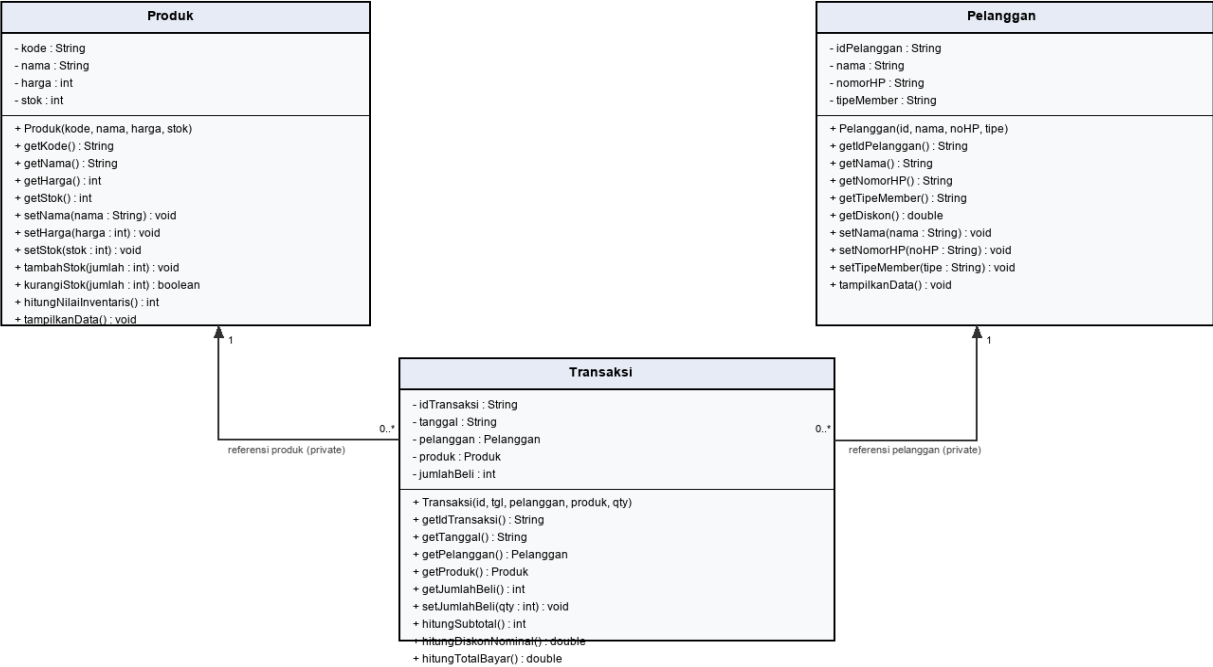
HALAMAN 2 — ENCAPSULATION & CLASS DIAGRAM

5. Tabel Spesifikasi Encapsulation, Getter, Setter, dan Validasi

Class	Private Attribute	Getter	Setter	Validation
Produk	• kode • nama • harga • stok	<code>getKode()</code> <code>getNama()</code> <code>getHarga()</code> <code>getStok()</code>	<code>setNama()</code> <code>setHarga()</code> <code>setStok()</code>	• harga > 0 • stok >= 0 • nama != null & non-blank
Pelanggan	• idPelanggan • nama • nomorHP • tipeMember	<code>getIdPelanggan()</code> <code>getNama()</code> <code>getNomorHP()</code> <code>getTipeMember()</code> <code>getDiskon()</code>	<code>setNama()</code> <code>setNomorHP()</code> <code>setTipeMember()</code>	• nama != null & non-blank • noHP: regex 10-13 digit • tipe: VIP, GOLD, REGULER
Transaksi	• idTransaksi • tanggal • pelanggan • produk • jumlahBeli	<code>getIdTransaksi()</code> <code>getTanggal()</code> <code>getPelanggan()</code> <code>getProduk()</code> <code>getJumlahBeli()</code>	<code>setJumlahBeli()</code>	• jumlahBeli > 0 • jumlahBeli <= produk.getStok()

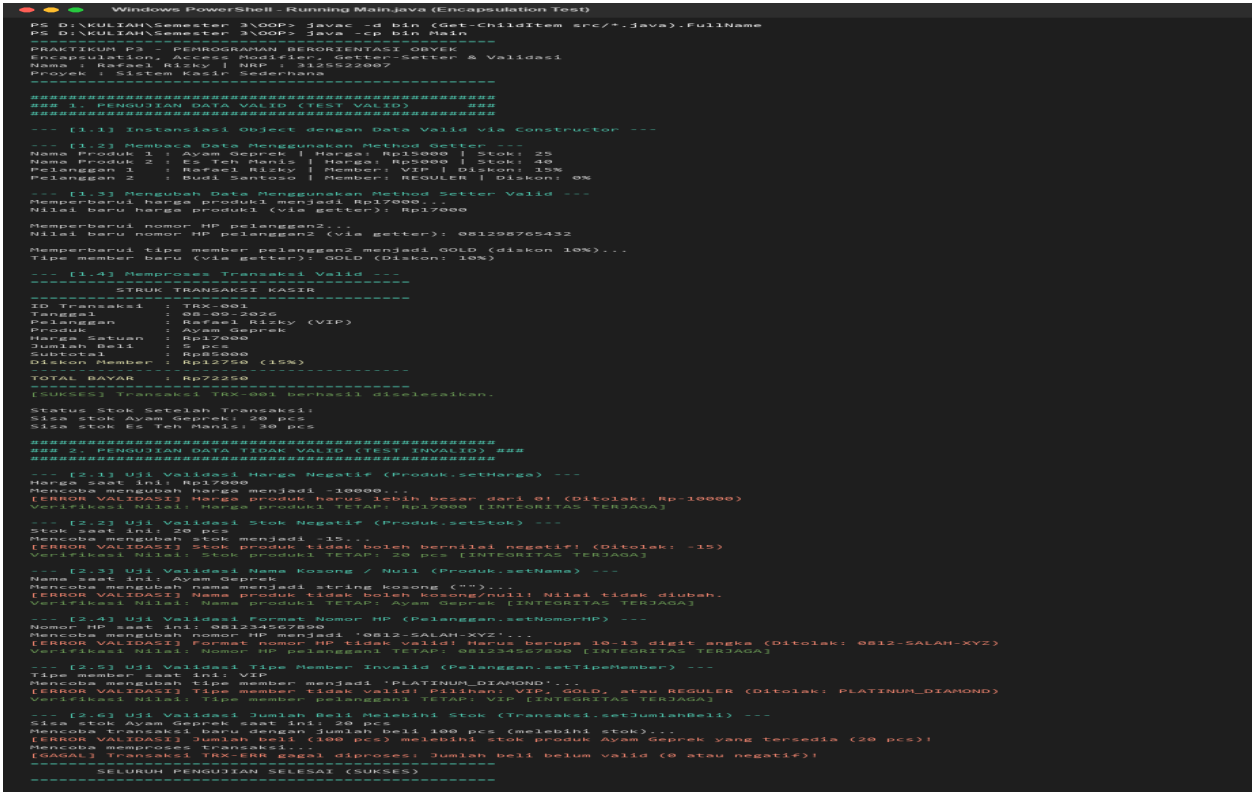
6. Diagram Kelas Hasil Refactoring (UML Class Diagram)

UML CLASS DIAGRAM (REFACTORED ENCAPSULATION) - P3



Gambar 1. UML Class Diagram Terenkapsulasi (Notasi - untuk Private dan + untuk Public)

7. Screenshot Hasil Running Program (Main.java)



Gambar 2. Bukti Running Program Main.java (Eksekusi Test Valid & Test Invalid)

8. Rincian Pengujian Valid dan Invalid:

- **Test Valid:** Instansiasi constructor valid, pembacaan via getter, pembaruan data setter (harga Rp17000, tipe member GOLD), serta checkout transaksi TRX-001 sukses mencetak struk dan memotong stok.
- **Test Invalid:** Uji harga negatif (-10000), stok negatif (-15), nama kosong (""), format no HP salah, tipe member salah, serta kuantitas melebihi stok berhasil ditolak dan data objek terlindungi utuh.

9. Sprint Review

Item	Hasil
Attribute berhasil di-private	Seluruh atribut class Produk, Pelanggan, dan Transaksi di-private.
Getter berjalan	Seluruh getter dapat membaca data tanpa membocorkan akses langsung.
Setter berjalan	Setter berjalan aman mengontrol nilai atribut yang diizinkan.
Validasi berhasil	Seluruh 6 aturan validasi berhasil menolak data yang tidak valid.
Constructor diperbaiki	Constructor memanggil setter sehingga validasi ditegakkan sejak inisialisasi.
Test invalid berhasil	Data tidak valid ditolak dan nilai atribut dalam objek tetap terjaga.
Program dapat dijalankan	Kompilasi (javac) dan eksekusi (java) berhasil 100% tanpa error.

10. Sprint Retrospective

What Went Well? Seluruh class berhasil dienkapsulasi dengan sempurna, information hiding terlindungi, dan validasi data terbukti kokoh menangkal input ilegal.

What Went Wrong? Penyesuaian method yang sebelumnya mengakses atribut langsung membutuhkan refactoring menyeluruh pada method pemanggil.

Improvement: Pada sprint berikutnya (P4), fokus dialihkan pada pemodelan relasi antarobject (asosiasi, agregasi, komposisi) yang lebih kaya dan dinamis.